
☑ Generics in Java — Full Notes with Examples

Generics allow you to create **classes, interfaces, and methods** that can operate on **any data type** while providing **compile-time type safety**.

◆ 1. Overview of Generics

✗ Without Generics (Old way)

```
ArrayList list = new ArrayList();  
list.add("Hello");  
list.add(10);    // Allowed (not type safe)
```

✓ With Generics

```
ArrayList<String> list = new ArrayList<>();  
list.add("Hello");  
// list.add(10);    // ✗ Compile-time error
```

Benefits

- **Type safety**
 - **Compile-time checking**
 - **No casting needed**
 - **Reusable code**
-

◆ 2. Simple Generic Class Example

```
class Box<T> {  
    private T value;  
  
    public void set(T value) {  
        this.value = value;  
    }  
}
```

```

        public T get() {
            return value;
        }
    }

    class Main {
        public static void main(String[] args) {
            Box<Integer> b1 = new Box<>();
            b1.set(100);
            System.out.println(b1.get());

            Box<String> b2 = new Box<>();
            b2.set("Hello");
            System.out.println(b2.get());
        }
    }
}

```

◆ 3. Multiple Generic Parameters

```

class Pair<K, V> {
    K key;
    V value;

    Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    void show() {
        System.out.println("Key: " + key + " Value: " + value);
    }
}

class Demo {
    public static void main(String[] args) {
        Pair<Integer, String> p = new Pair<>(1, "One");
        p.show();
    }
}

```

◆ 4. Bounds (Upper Bound & Lower Bound)

👉 **Upper Bound:** `<T extends ClassName>`

Allows only that type or its subclasses.

```

class Data<T extends Number> {
    T num;

    Data(T num) {
        this.num = num;
    }

    void show() {
        System.out.println(num);
    }
}

class Main {
    public static void main(String[] args) {
        Data<Integer> d1 = new Data<>(10);
        Data<Double> d2 = new Data<>(10.5);

        // Data<String> d3 = new Data<>("Hello"); // ✗ Not allowed
    }
}

```

👉 **Lower Bound: <? super ClassName>**

Useful for writing into lists.

```

public static void addNumbers(List<? super Integer> list) {
    list.add(10);
    list.add(20);
}

```

◆ 5. Wildcards

1. Unbounded Wildcard <?>

Used when type is unknown.

```

public static void printList(List<?> list) {
    for(Object o : list) {
        System.out.println(o);
    }
}

```

2. Upper Bounded Wildcard <? extends Number>

```

public static void print(List<? extends Number> list) {
    for(Number num : list) {
        System.out.println(num);
    }
}

```

```
}
```

3. Lower Bounded Wildcard <? super Integer>

```
public static void addValues(List<? super Integer> list) {  
    list.add(10);  
    list.add(20);  
}
```

6. Generic Methods

```
class Helper {  
    public static <T> void display(T value) {  
        System.out.println(value);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Helper.display(100);  
        Helper.display("Hello");  
    }  
}
```

7. Generic Interfaces

```
interface Container<T> {  
    void show(T data);  
}  
  
class Box<T> implements Container<T> {  
    public void show(T data) {  
        System.out.println("Showing: " + data);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Box<String> b = new Box<>();  
        b.show("Generics in Java");  
    }  
}
```

8. Generic Classes (Advanced Example)

```

class Store<T> {
    T[] items;

    Store(T[] items) {
        this.items = items;
    }

    public void display() {
        for(T item : items) {
            System.out.println(item);
        }
    }
}

class Main {
    public static void main(String[] args) {
        Integer[] arr = {1, 2, 3, 4};
        Store<Integer> s = new Store<>(arr);
        s.display();
    }
}

```



Final Summary Table

Topic	Meaning	Example
Generic Class	Class with type parameter	<code>class Box<T></code>
Generic Interface	Interface with type parameter	<code>interface Container<T></code>
Generic Method	Method with <code><T></code>	<code><T> void show(T x)</code>
Multiple Parameters	More than one type	<code>class Pair<K,V></code>
Upper Bound	<code>T extends Number</code>	Only numbers
Lower Bound	<code>? super Integer</code>	Integers + super types
Wildcards	Unknown type	<code>List<?></code>
